



CHAPTER-8-ARTIFICIAL NEURAL NETWORK

Wattanapong Suttapak

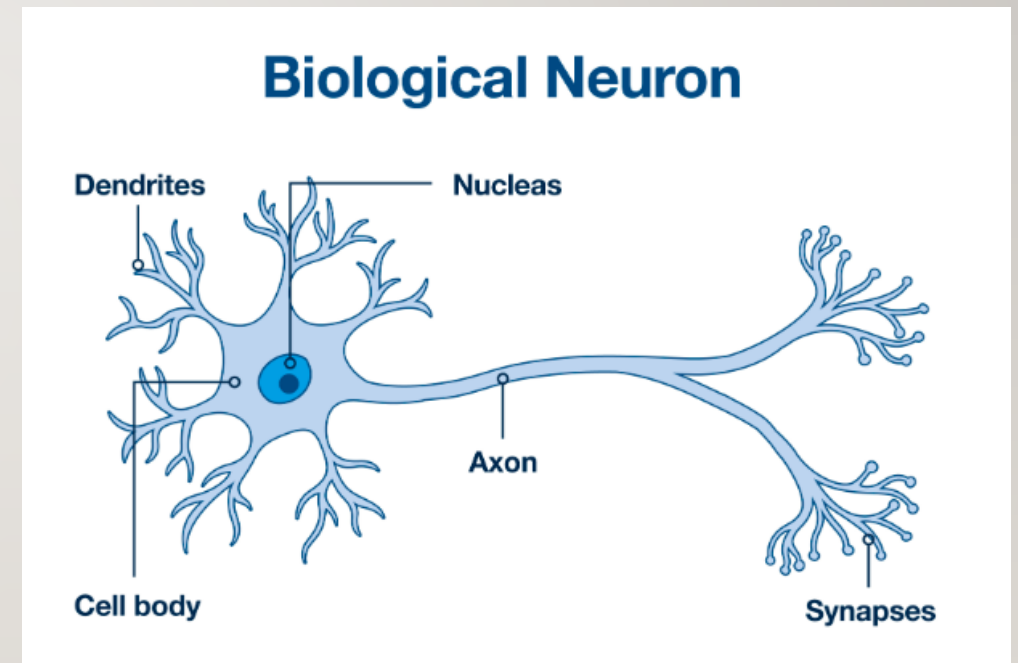


2 OUTLINE

- Definition of Artificial Neural Networks
- Basic Structure of Artificial Neural Networks
- Activation Functions
- Training Neural Networks
- Types of Neural Networks
- Applications of Artificial Neural Networks
- Challenges and Considerations
- Tools and Frameworks
- Future Trends in Artificial Neural Networks

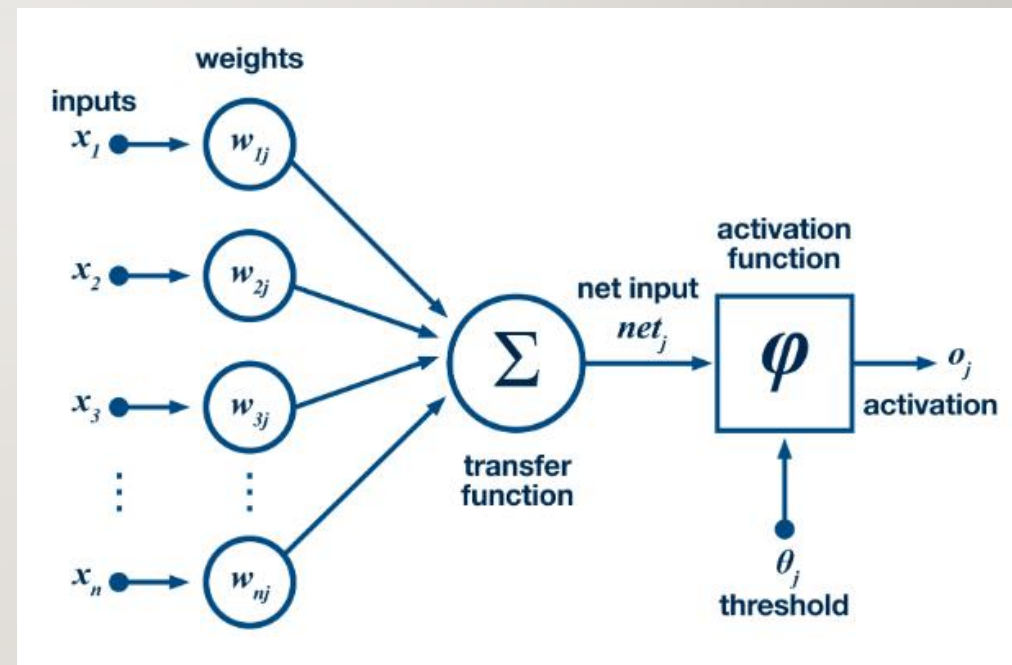
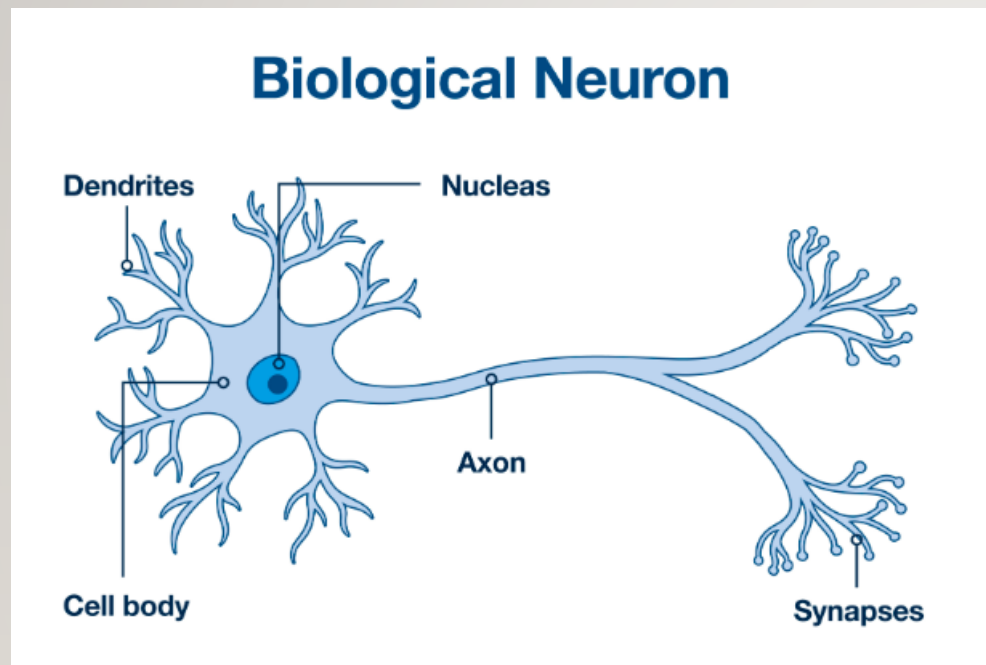
3 DEFINITION OF ARTIFICIAL NEURAL NETWORKS

- ได้รับแรงบันดาลใจจากโครงสร้างสมองมนุษย์
- ถูกสร้างขึ้นในปี ค.ศ. 1940
- พัฒนา **Backpropagation** ในปี 1960s
- เลียนแบบระบบประสาทของมนุษย์
 - หนึ่ง **Neuron** ประกอบด้วย **Dendrites**
 - แต่ละ **Dendrite** ทำหน้าที่รับสัญญาณ
 - แต่ละ **Axon** ทำหน้าที่ส่งสัญญาณไปยัง **neurons** อื่น ๆ



[Ref: Introduction to neural networks – Electronics World](#)

4 DEFINITION OF ARTIFICIAL NEURAL NETWORKS



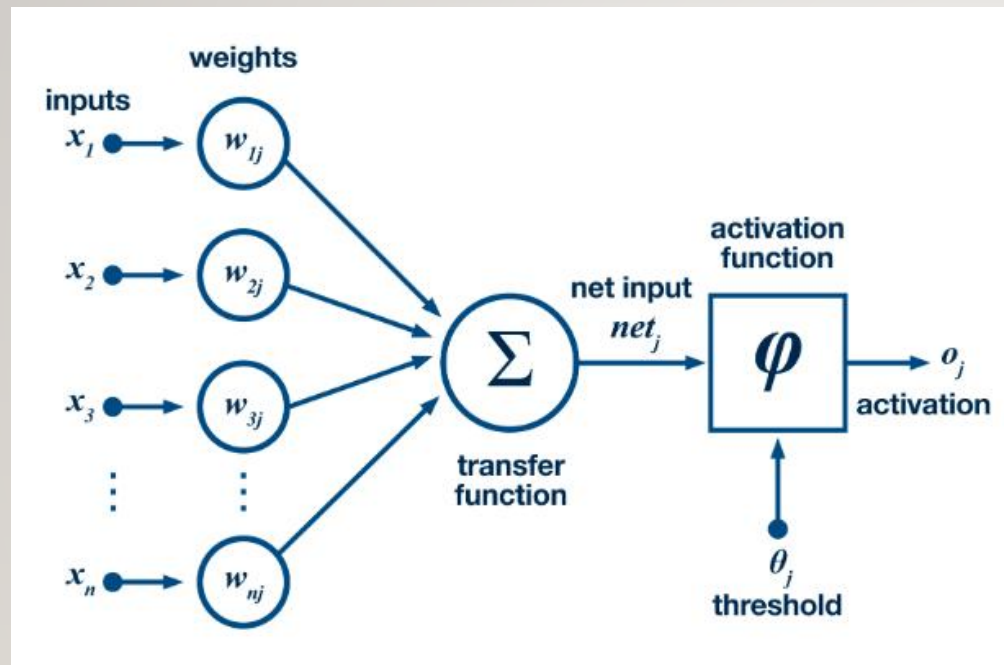
- Node $W = \{w_{11}, w_{12}, \dots, w_{ij}\}$
- Transfer Function เป็นฟังก์ชันที่นำข้อมูลของแต่ละ Node มาทำงานร่วมกัน
- Activation function เป็นฟังก์ชันตัดสินใจว่า Output ควรเป็นอะไร

5

BASIC STRUCTURE OF ARTIFICIAL NEURAL NETWORKS



6 BASIC STRUCTURE OF ARTIFICIAL NEURAL NETWORKS



- Node $W = \{w_1, w_2, \dots, w_j\}$
- Transfer Function เป็นฟังก์ชันที่นำข้อมูลของแต่ละ Node มาทำงานร่วมกัน
- Activation function เป็นฟังก์ชันตัดสินใจว่า Output ควรเป็นอะไร

$$Y' = f(X) + \beta \quad \text{for } X \text{ is input}$$

$f(.)$ is transfer function and activation function

β is bias (ค่าความผิดพลาด Error ที่ยอมรับได้)

Loss function is $\min|Y - Y'| \rightarrow 0$

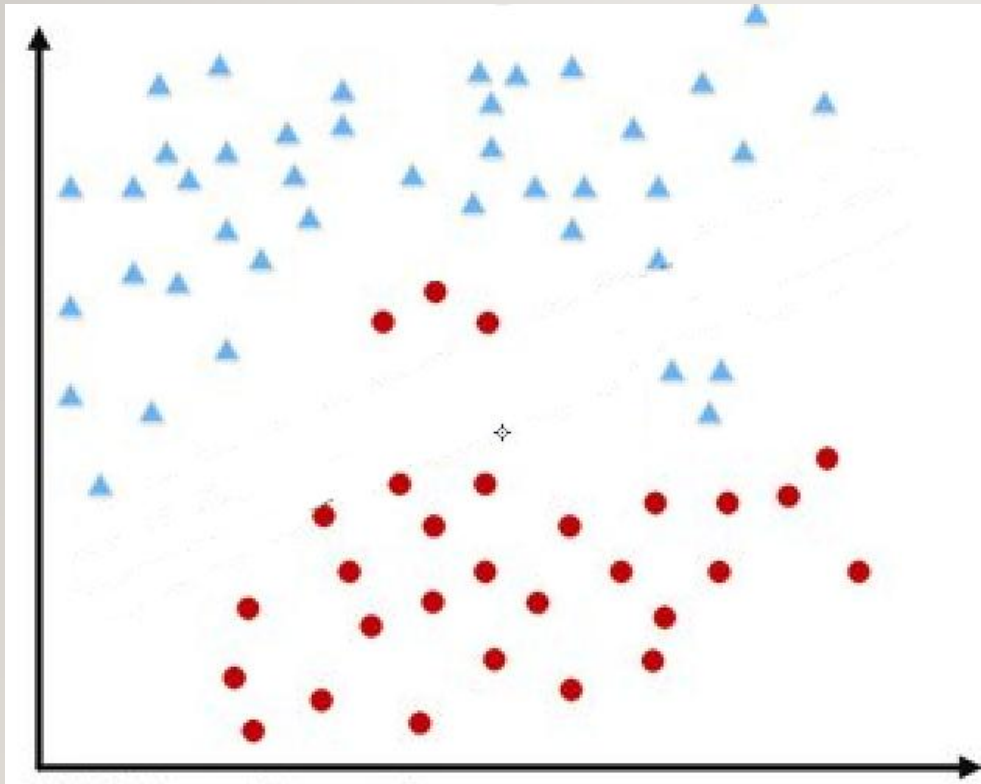
7 BASIC STRUCTURE OF ARTIFICIAL NEURAL NETWORKS

- Weight Update:

$$w_i = w_i - \alpha \frac{\partial L}{\partial w_i}$$

- α is learning rate (เป็นขนาดของ step ในการเรียนรู้ เมื่อเพิ่มค่า lr การเปลี่ยนแปลงค่าจะมากขึ้น หากลดค่า lr การเปลี่ยนแปลงค่าการเรียนรู้จะช้าลง)
- $\frac{\partial L}{\partial w_i}$ isอนุพันธ์ (partial differentiate) ของ loss function

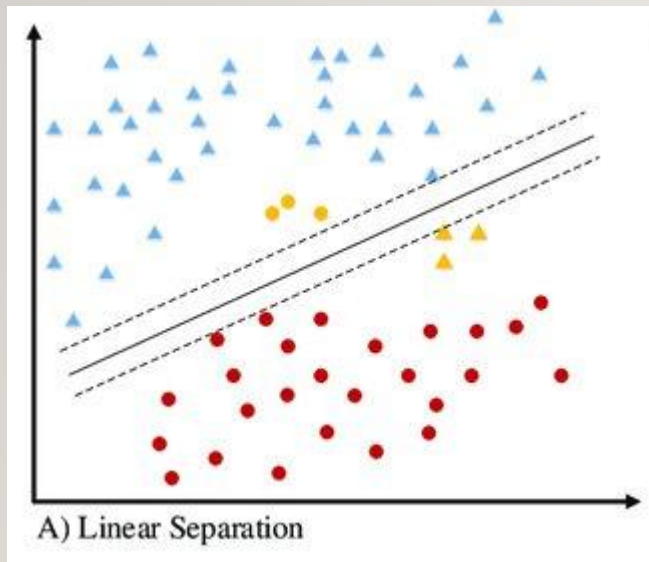
8 DATASET CHARACTERISTICS



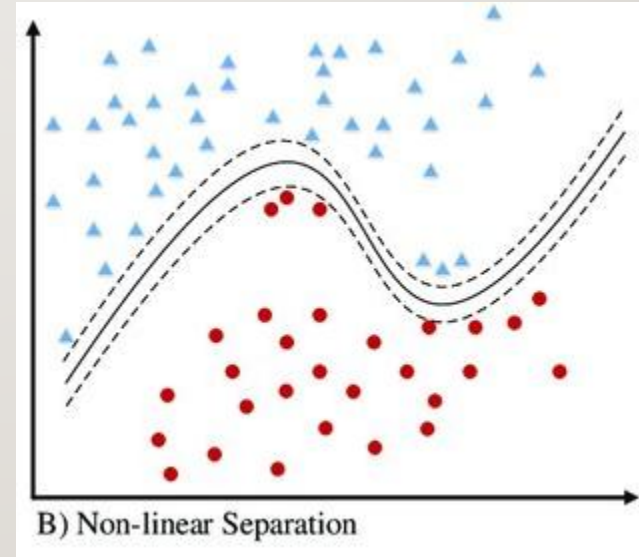
- เราสามารถหาเส้นแบ่งระหว่างวงกลมสีแดง กับ สามเหลี่ยมสีฟ้าได้หรือไม่

9 DATASET CHARACTERISTICS

LINEAR



NON-LINEAR



Kowsari, Kamran, et al. "Text classification algorithms:A survey." *Information* 10.4 (2019): 150.

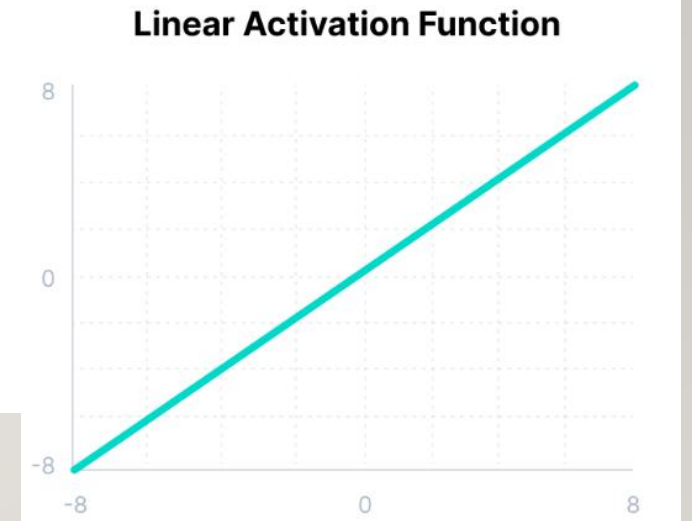
10 ACTIVATION FUNCTIONS

- Linear function
- Sigmoid function
- Hyperbolic tangent function
- ReLU function
- Leaky ReLU function
- Softmax function

II LINEAR FUNCTION

Linear Activation (Identity Function):

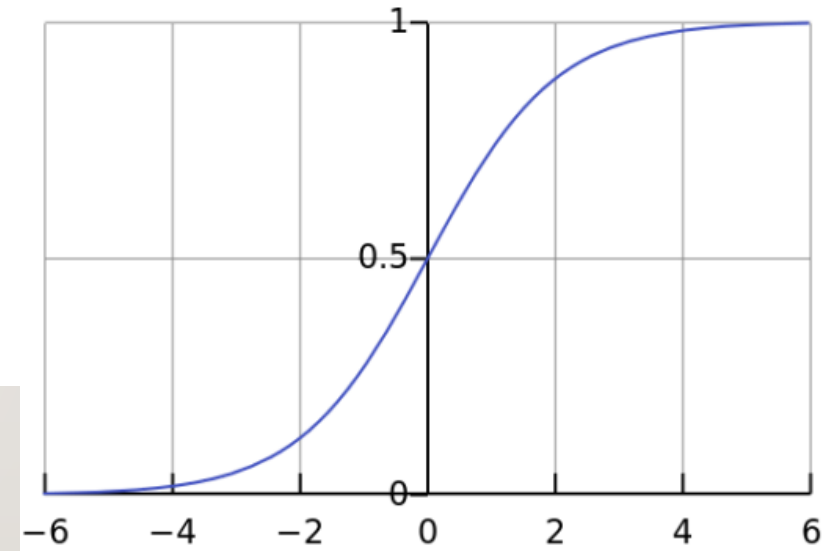
- Formula: $f(x) = x$
- Characteristics: This is a straightforward identity function, and it produces a linear output. It is suitable for regression tasks or cases where the network needs to predict continuous values.



I2 SIGMOID FUNCTION

Sigmoid Function (Logistic Function):

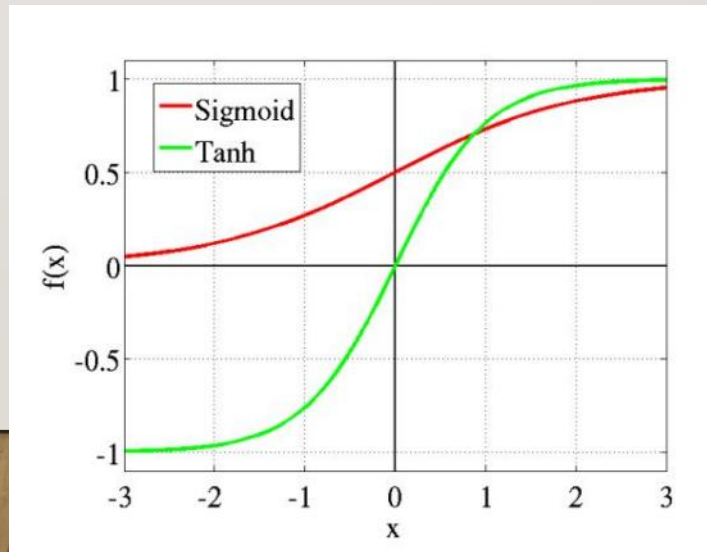
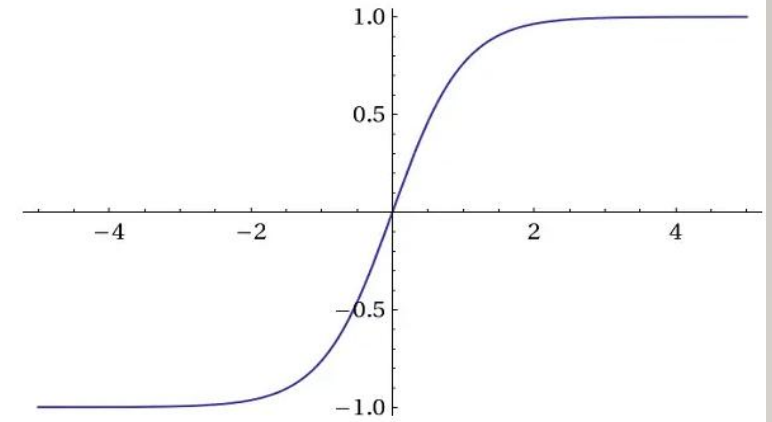
- Formula: $f(x) = \frac{1}{1+e^{-x}}$
- Output Range: (0, 1)
- Characteristics: Smooth and differentiable, commonly used in the output layer for binary classification problems.



13 HYPERBOLIC TANGENT FUNCTION

Hyperbolic Tangent Function (tanh):

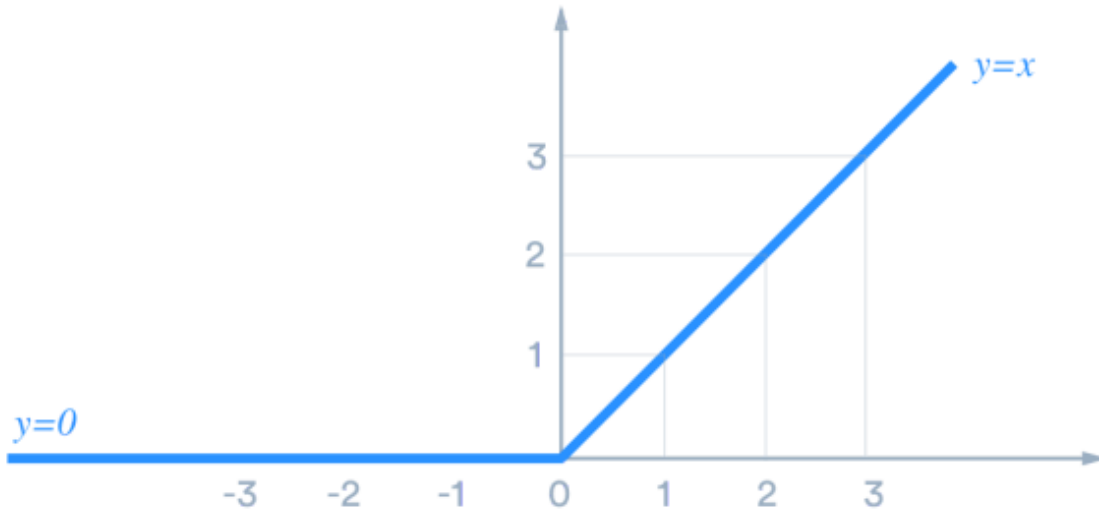
- Formula: $f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
- Output Range: $(-1, 1)$
- Characteristics: Similar to the sigmoid function but with an extended range, often used in hidden layers.



14 RELU

Rectified Linear Unit (ReLU):

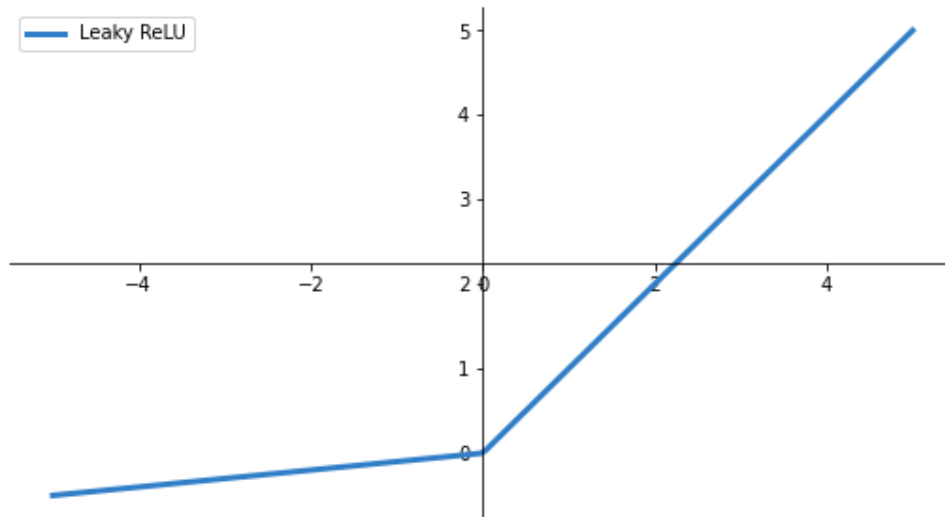
- Formula: $f(x) = \max(0, x)$
- Output Range: $[0, +\infty)$
- Characteristics: Piecewise linear, computationally efficient, and helps mitigate the vanishing gradient problem.



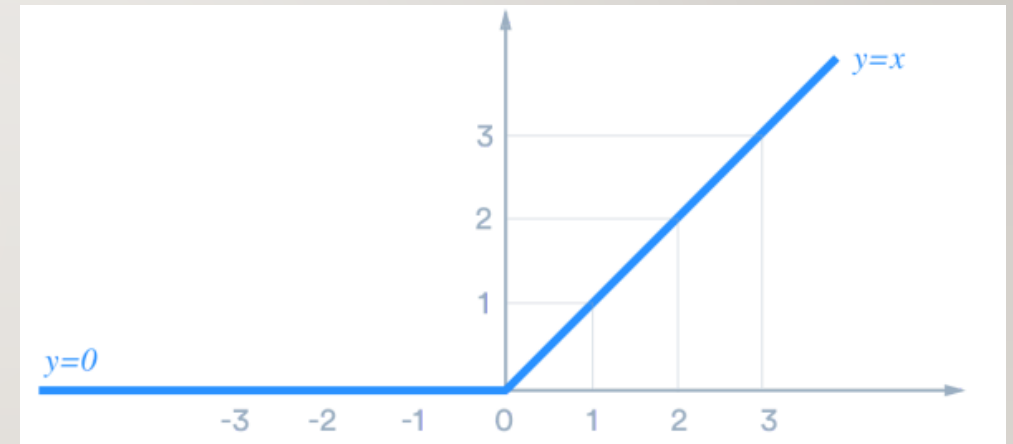
15 LEAKY RELU

Leaky Rectified Linear Unit (Leaky ReLU):

- Formula: $f(x) = \max(\alpha x, x)$ where α is a small positive constant (e.g., 0.01).
- Output Range: $(-\infty, +\infty)$
- Characteristics: Similar to ReLU but allows a small, non-zero gradient for negative inputs, addressing the dying ReLU



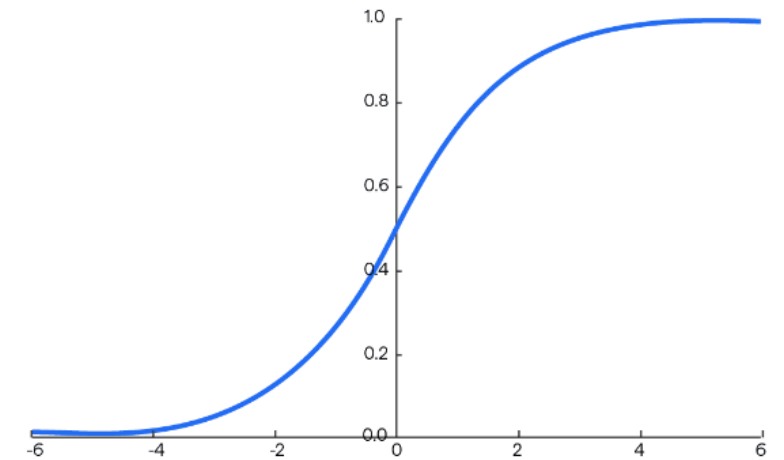
ReLU



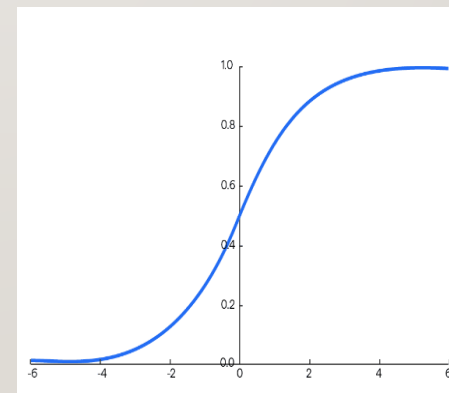
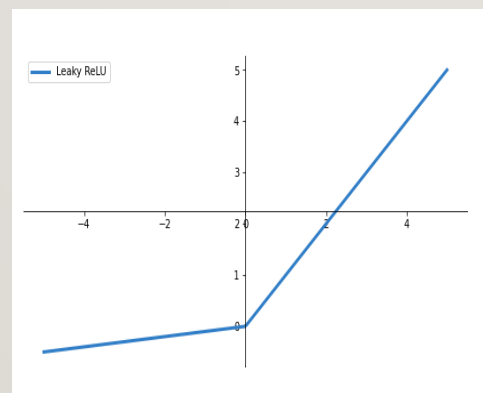
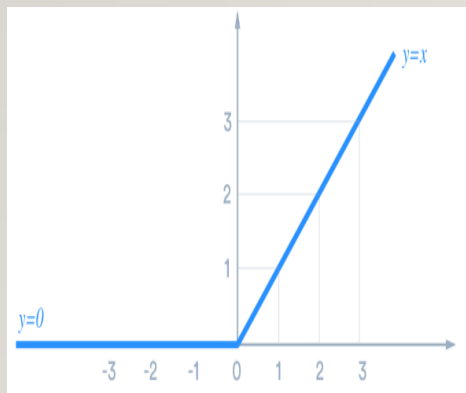
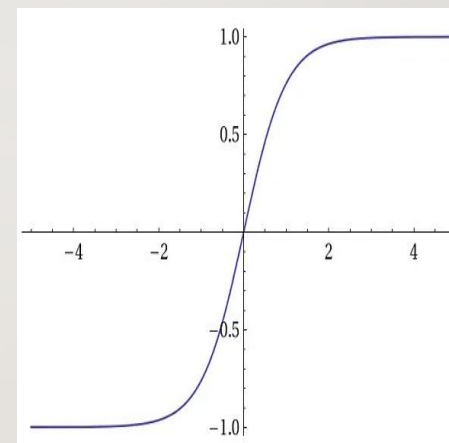
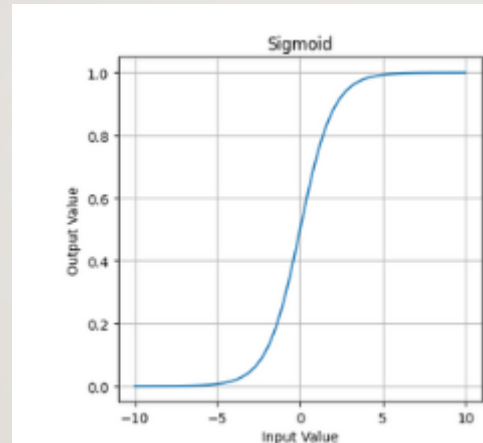
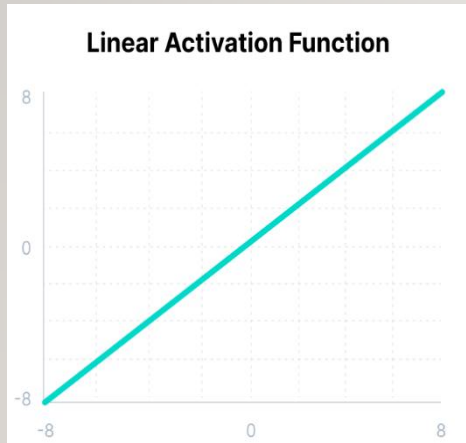
16 SOFTMAX FUNCTION

Softmax Function:

- Formula: $f(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$ for each x_i in the output layer.
- Output Range: (0, 1), with the sum of all outputs equal to 1.
- Characteristics: Often used in the output layer for multi-class classification problems, as it converts raw scores into probabilities.



17 ACTIVATION FUNCTION

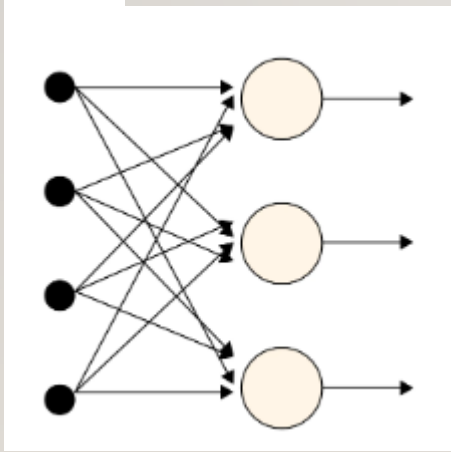


18 TYPE OF NEURAL NETWORKS

Feedforward Neural Networks (FNN)

- โครงสร้างเป็นแบบไม่เรียงง่าย ไม่ซับซ้อน มีทิศทางการไหลของข้อมูลในทิศทางเดียว จาก Input ไปสู่ Output
- ใช้ในงาน **Classification** โดยสามารถแยกย่อยเป็น **SLP (SINGLE LAYER PERCEPTRON)** และ **MLP (MULTI LAYER PERCEPTRON)**

FNN

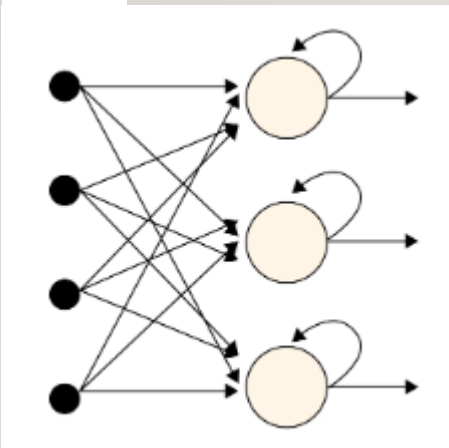


19 TYPE OF NEURAL NETWORKS

Recurrent Neural Networks (RNN)

- เพิ่มความสามารถจาก FNN โดยยอมให้ Node ($\text{Input} * \text{Weight}$) แต่ละ Node สามารถส่งข้อมูลย้อนกลับมาที่ Function เดิมได้ (Cycle)
- ใช้ในงานที่ข้อมูลมีลักษณะเป็นลำดับ sequential data ยกตัวอย่างเช่น **time series prediction**

RNN

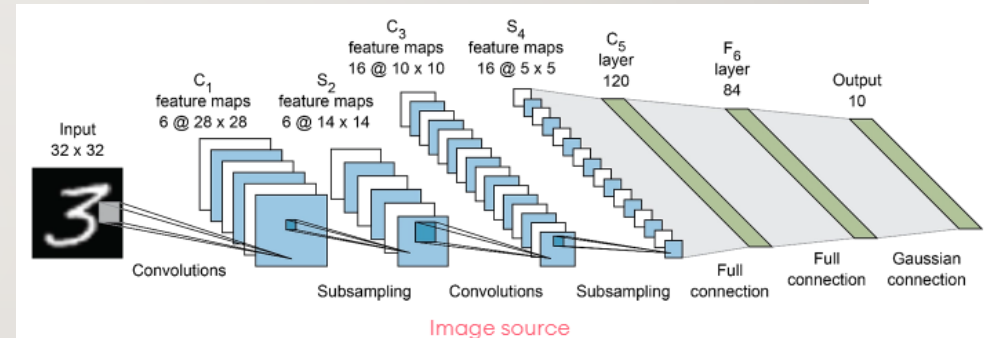


20 TYPE OF NEURAL NETWORKS

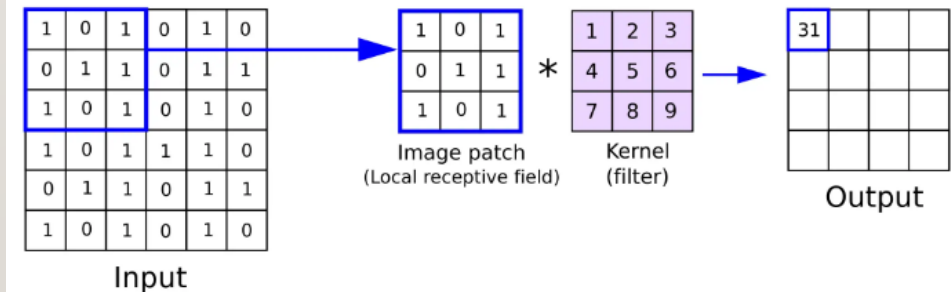
Convolutional Neural Networks (CNN)

- โครงสร้างข้อมูลมักอยู่ในรูปแบบ 2 มิติ เช่น รูปภาพ จะถูกส่งผ่านไปยัง convolutional layers
- ใช้ในงาน **Image Processing** และ **Computer Vision**

CNN

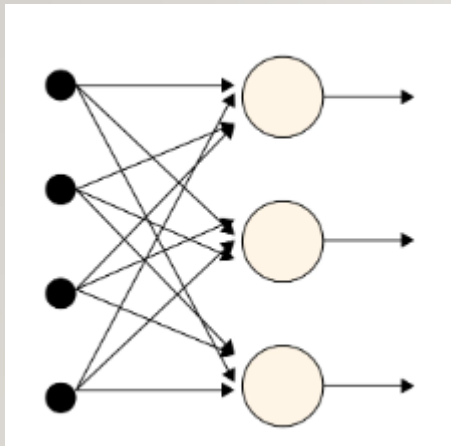


The convolutional layer

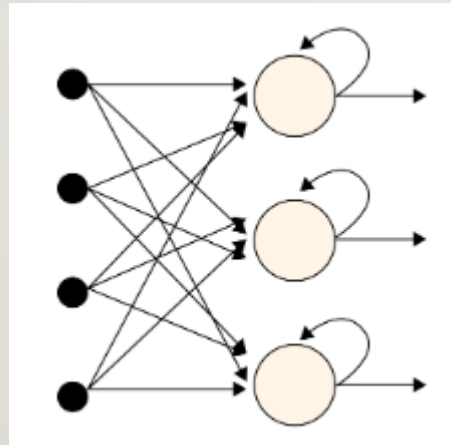


21 TYPE OF NEURAL NETWORKS

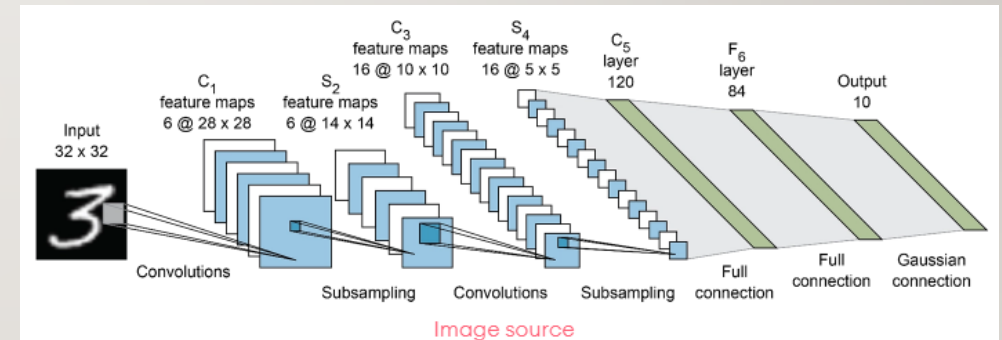
FNN



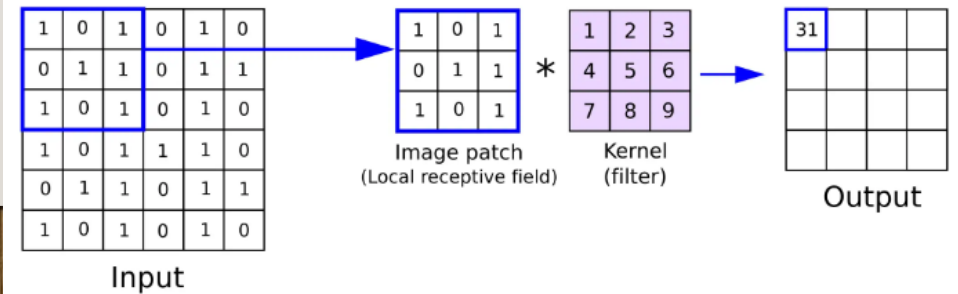
RNN



CNN



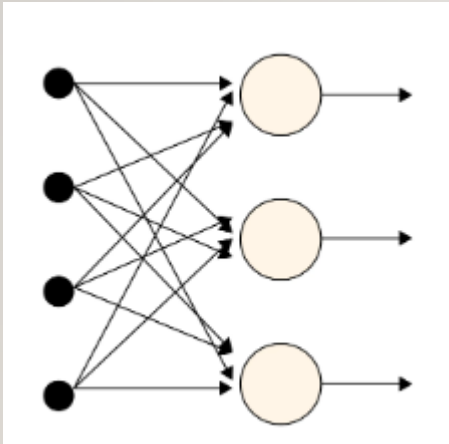
The convolutional layer



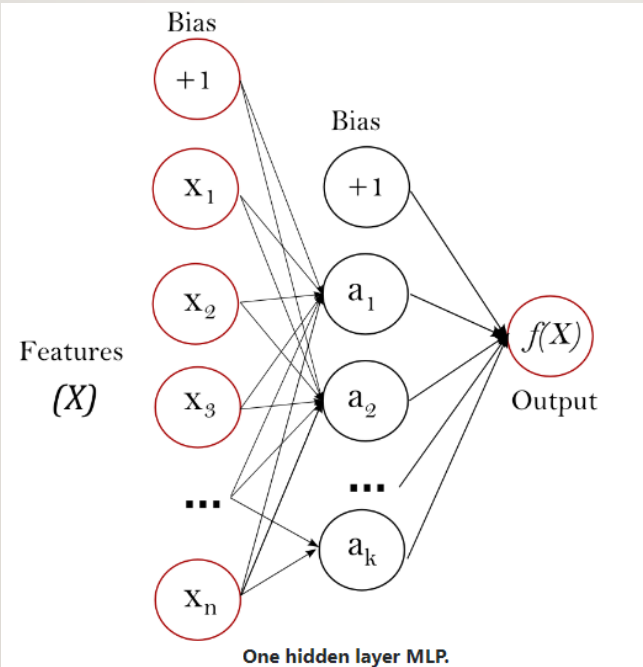
22 TYPE OF NEURAL NETWORKS

MLP เป็นลักษณะของ FNN โดยเพิ่มจำนวนโหนด และ Layer มากขึ้น จากเดิม FNN มีเพียง Layer เดียว

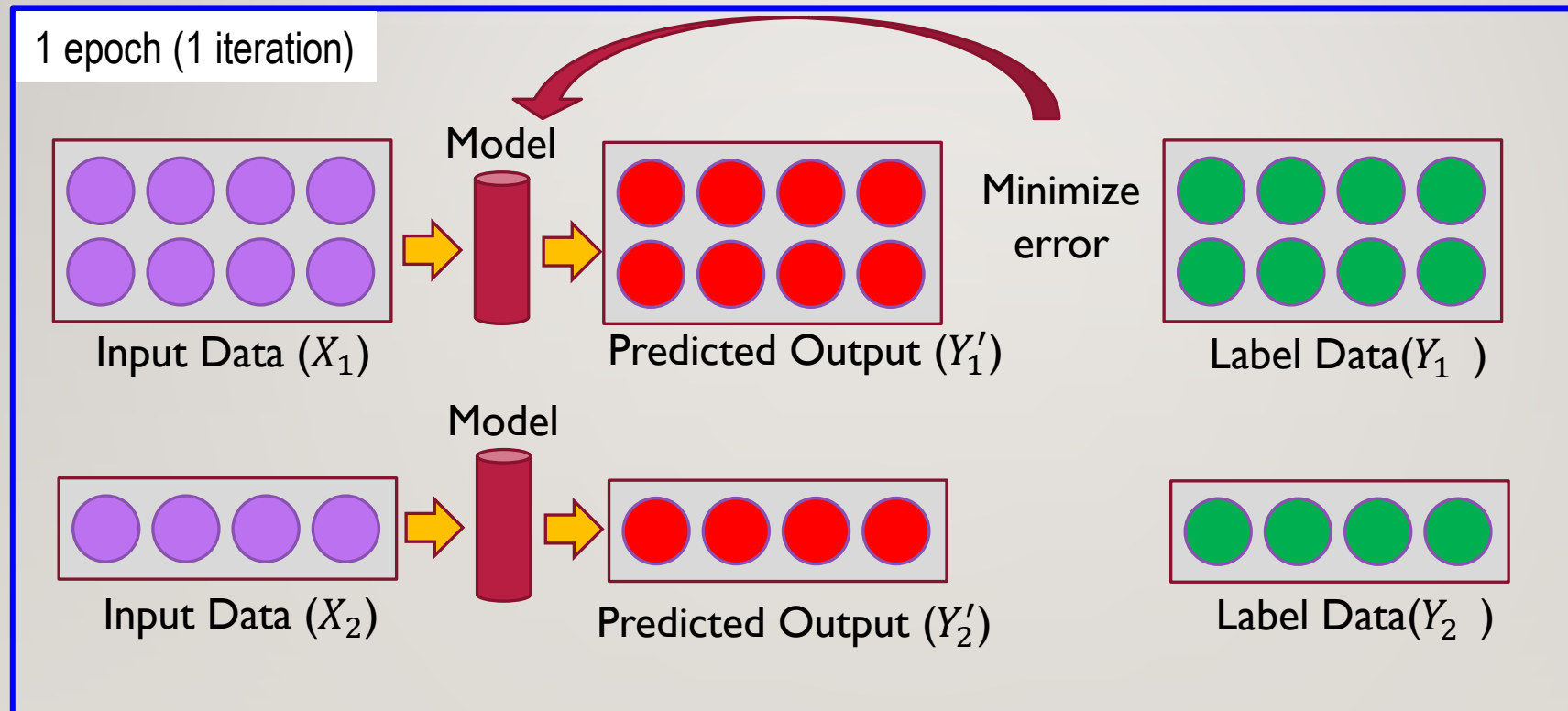
FNN



MLP (Multilayer perceptron)



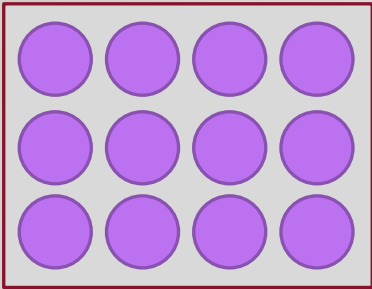
23 TRAINING NEURAL NETWORKS



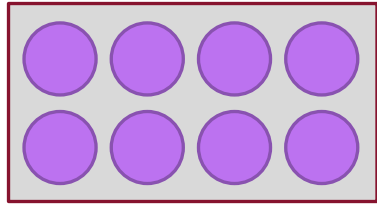
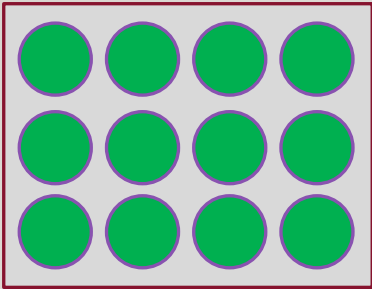
Repeat
1 – n epochs
Or
Converged
(error $\rightarrow 0$)

24 TRAINING NEURAL NETWORKS

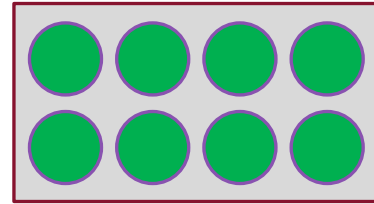
Input Data (X)



Label Data (Y)

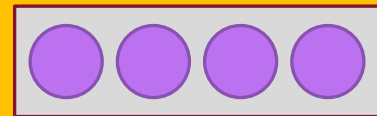


Input Data (X_1)



Label Data (Y_1)

Train Data



Input Data (X_2)



Label Data (Y_2)

Validation Data

25 SINGLE LAYER PERCEPTRON

- Classification
 - Binary Classification (2 classes)
 - Multiclass Classification (> 2 classes)
- Regression
 - Predict output based on input

26 SINGLE LAYER PERCEPTRON

BINARY CLASSIFICATION

- กำหนดให้ $\mathbf{X}$ แทนสัญลักษณ์ข้อมูล
- $\mathbf{X} \in \mathbb{R}^{n \times m}$ โดยที่ n เป็นจำนวนของข้อมูลทั้งหมด m เป็นจำนวนลักษณะเด่น (Feature)
- $\mathbf{w}$ เป็นเวกเตอร์ค่าถ่วงน้ำหนักมีขนาดเท่ากับ จำนวน Feature = m
- $\mathbf{b}$ เป็นเวกเตอร์ Bias เพื่อชดเชยค่าผลลัพธ์ในบางค่า มีขนาดเท่ากับจำนวนของ n

$$\mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & \dots & x_{1m} \\ x_{21} & x_{22} & \dots & x_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \dots & x_{nm} \end{bmatrix}$$

$$\mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_m \end{bmatrix}$$

$$\mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

$$\mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

27 SINGLE LAYER PERCEPTRON BINARY CLASSIFICATION

- ในแต่ละ **sample** สามารถคำนวณค่าของผลลัพธ์ **z** ได้ดังนี้

$$z_i = \mathbf{x}_i \cdot \mathbf{w} + b$$

- ทั้งนี้เราสามารถคำนวณทุก **sample** ได้เลย โดยการใช้คุณสมบัติของ **matrix** ในการคูณกัน

$$\mathbf{Z} = \mathbf{w}^T \mathbf{X} + b$$

28 SINGLE LAYER PERCEPTRON

BINARY CLASSIFICATION

- สำหรับ **Binary classification** เราสามารถใช้ **Activation function** เป็น **Step function** ดังนี้

$$y_i = \begin{cases} 1 & \text{if } z_i \geq 0 \\ 0 & \text{if } z_i < 0 \end{cases}$$

- **Predicted output** สามารถหาได้จากการคำนวณของ **Activation function**

29 SINGLE LAYER PERCEPTRON BINARY CLASSIFICATION

การอัปเดตค่า **weight** สามารถหาได้จากวิธีการคำนวณค่าความผิดพลาดที่เกิดขึ้นดังนี้

$$e_i = t_i - y_i$$

กำหนดให้ e_i เป็นค่าความผิดพลาดระหว่าง t_i และ y_i โดย t_i เป็นผลลัพธ์เป้าหมาย และ y_i เป็น **predicted output**

ทำการอัปเดตค่า **w** โดยใช้สมการดังนี้ กำหนดให้ เอต้า (η) เป็น **learning rate**

$$\mathbf{w} \leftarrow \mathbf{w} + \eta \cdot e_i \cdot \mathbf{x}_i$$

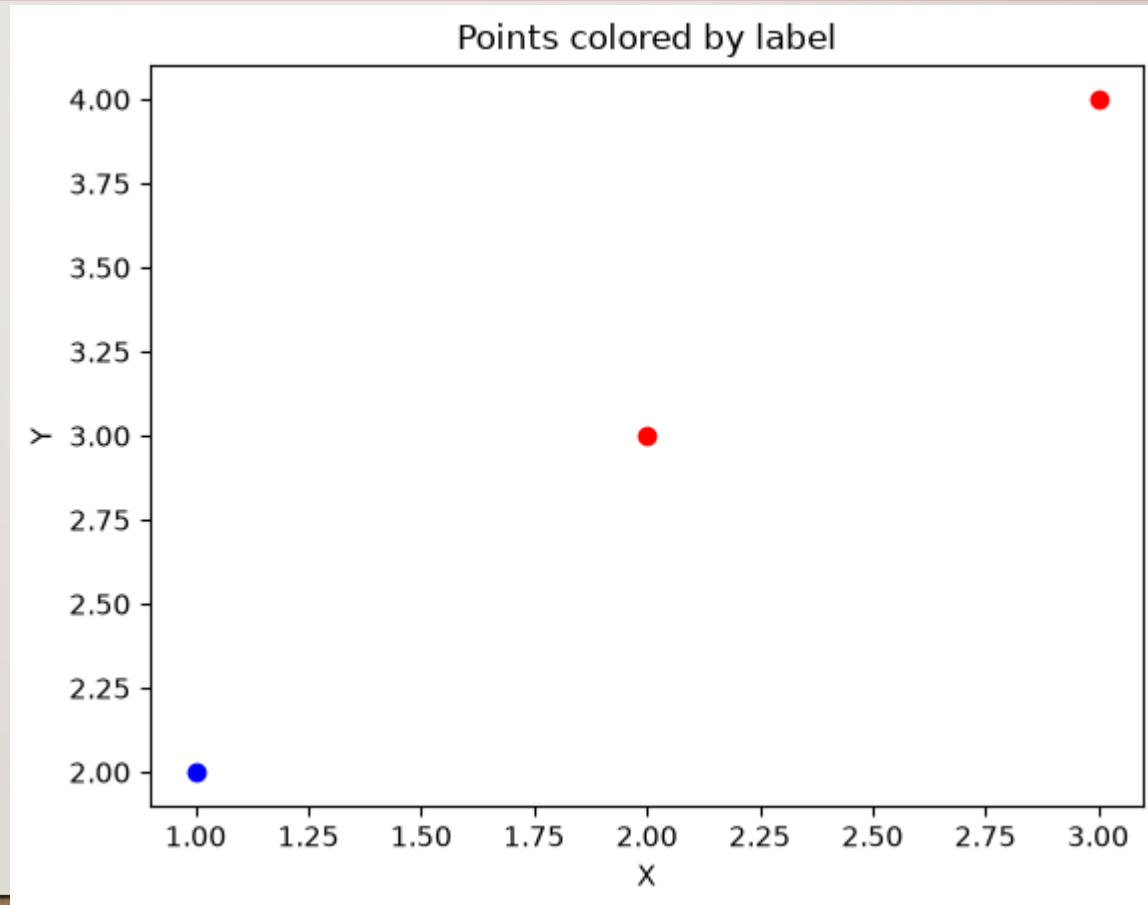
ทำการคำนวณซ้ำทุก ๆ sample

30 SINGLE LAYER PERCEPTRON

BINARY CLASSIFICATION

- ตัวอย่างการคำนวณ

- Input Data (x): $X = \begin{bmatrix} 1 & 2 \\ 2 & 3 \\ 3 & 4 \end{bmatrix}$
- True Labels (y): $y = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}$



32 SINGLE LAYER PERCEPTRON

BINARY CLASSIFICATION

- Input Data ($\mathbf{X}$): $\mathbf{X} = \begin{bmatrix} 1 & 2 \\ 2 & 3 \\ 3 & 4 \end{bmatrix}$
- True Labels ($\mathbf{y}$): $\mathbf{y} = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}$
- Initial Weights ($\mathbf{w}$): $\mathbf{w} = \begin{bmatrix} 0.5 \\ 1 \end{bmatrix}$
- Initial Bias (b): $b = 0$
- Learning Rate (η): $\eta = 0.1$

Step 1: Compute the Linear Combination (Activation Input)

คำนวณค่าผสมการแบ่งข้อมูล ด้วย initial weight และ initial bias

$$z_i = \mathbf{x}_i \cdot \mathbf{w} + b$$

For sample 1 ($i = 1$):

$$z_1 = (1 \times 0.5) + (2 \times 1) + 0 = 0.5 + 2 = 2.5$$

For sample 2 ($i = 2$):

$$z_2 = (2 \times 0.5) + (3 \times 1) + 0 = 1 + 3 = 4$$

For sample 3 ($i = 3$):

$$z_3 = (3 \times 0.5) + (4 \times 1) + 0 = 1.5 + 4 = 5.5$$

33 SINGLE LAYER PERCEPTRON

BINARY CLASSIFICATION

Step 2: Apply the Activation Function (Step Function)

คำนวณผลลัพธ์ที่ได้จากสมการ การแบ่งข้อมูล

$$y_{\text{pred}_i} = \begin{cases} 1 & \text{if } z_i > 0 \\ 0 & \text{if } z_i \leq 0 \end{cases}$$

For sample 1 ($i = 1$):

$$y_{\text{pred}_1} = 1 \quad (\text{since } z_1 = 2.5 > 0)$$

For sample 2 ($i = 2$):

$$y_{\text{pred}_2} = 1 \quad (\text{since } z_2 = 4 > 0)$$

For sample 3 ($i = 3$):

$$y_{\text{pred}_3} = 1 \quad (\text{since } z_3 = 5.5 > 0)$$

34 SINGLE LAYER PERCEPTRON

BINARY CLASSIFICATION

Step 3: Calculate the Error

คำนวณค่า **error** จากผลลัพธ์แต่ละ sample

$$\text{Error}_i = y_i - y_{\text{pred}_i}$$

For sample 1 ($i = 1$):

$$\text{Error}_1 = 0 - 1 = -1$$

For sample 2 ($i = 2$):

$$\text{Error}_2 = 1 - 1 = 0$$

For sample 3 ($i = 3$):

$$\text{Error}_3 = 1 - 1 = 0$$

35 SINGLE LAYER PERCEPTRON BINARY CLASSIFICATION

Step 4: Update the Weights and Bias

อัปเดต weight และ bias (Option)

$$\mathbf{w} \leftarrow \mathbf{w} + \eta \times \text{Error}_i \times \mathbf{x}_i$$

$$b \leftarrow b + \eta \times \text{Error}_i$$

$$\begin{aligned} \mathbf{w}_0 &= \mathbf{w}_0 + 0.001 * 1 * [1 \ 2] \\ &= [0.5 \ 1] + 0.001 * [1 \ 2] \\ &= [0.5001 \ 1.0002] \end{aligned}$$

$$\begin{aligned} b_0 &= b_0 + 0.001 * 1 \\ &= 0.001 \end{aligned}$$

Repeat sample 1 and 2 with $\mathbf{w}_0$ and b_0

Repeat step 1 – 4 until Error is 0

36 SINGLE LAYER PERCEPTRON

MULTICLASS CLASSIFICATION

1. คำนวณผลลัพธ์แต่ละ sample ที่ i

$$z_{ik} = \mathbf{x}_i \cdot \mathbf{w}_k + b_k$$

2. คำนวณค่า **Activation function** ด้วยการใส่ **Softmax function** (สำหรับ **Multiclass classification**)

$$\text{Softmax}(z_i)_k = \frac{e^{z_{ik}}}{\sum_{j=1}^C e^{z_{ij}}}$$

3. คำนวณค่า **Predicted output** โดยการหาค่า **maximum index** ของ **Softmax function**

$$\hat{y}_i = \arg \max_k \text{Softmax}(z_i)_k$$

4. คำนวณค่า **Loss function**

$$L = - \sum_{i=1}^n \sum_{k=1}^C t_{ik} \log(\text{Softmax}(z_i)_k)$$

5. อัปเดต **weight**

$$\mathbf{w}_k \leftarrow \mathbf{w}_k - \eta \cdot \frac{\partial L}{\partial \mathbf{w}_k}$$

6. **Gradient Descent**

$$\frac{\partial L}{\partial \mathbf{w}_k} = -\frac{1}{n} \sum_{i=1}^n (t_{ik} - \text{Softmax}(z_{ik})) \mathbf{x}_i$$